Chapter 8

Stochastic search methods

Chapter 8. Stochastic search methods for CSPs

8.1 Introduction

8.2 Hill-climbing

8.3 Connectionist Approach

8.1 Introduction

In many situations, a timely response by a CSP solver is crucial. For example, some CSPs may take days or years to solve with conventional hardware using the complete search methods so far introduced in this book. In applications such as industrial scheduling, the user may like to analyse a large number of hypothetical situations. For such applications, the user may need to evaluate the effect of different combinations of constraints, and therefore speed in a CSP solver is important.

Stochastic search is a class of search methods which includes heuristics and an element of nondeterminism in traversing the search space. Unlike the search algorithms introduced so far, a stochastic search algorithm moves from one point to another in the search space in a nondeterministic manner, guided by heuristics. The next move is partly determined by the outcome of the previous move. Stochastic search algorithms are, in general, incomplete.

8.2 Hill-climbing

8.2.1 General hill-climbing algorithms

The general hill-climbing algorithm requires two functions: an *evaluation function* which maps every point in the search space to a value (which is a number), and an *adjacency function* which maps every point in the search space to other points. The solution is the point in the search space that has the greatest value according to the evaluation function.

Hill-climbing algorithms normally start with a random focal point in the search space. Given the current focal point P, all the points which are adjacent to P are evaluated. If some points have greater values than P's, then one of these will be picked non-deterministically to become the new focal point. Heuristics can be used for choosing from among the higher points. The algorithm continues until the value of the current focal point is greater than the values of all the nodes adjacent to it. The current focal point is either a solution or a local maximum.

```
PROCEDURE Generic_Hill_Climbing(e,c)
/* Given a point P in the search space, e(P) maps P to a numerical
    value which is to be maximized; c maps any point P to a (possibly
    empty) set of points in the search space*/
BEGIN
    /* Initialization */
    P ← random point in the search space;
    SP ← c(P);          /* SP is the set of points adjacent to P */

    /* Hill-climbing */
    WHILE (there exists a point Q in SP such that e(Q) ≥ e(P)) DO
        BEGIN
            Q ← a point in SP such that e(Q) ≥ e(P);
            /* heuristics may be used here in choosing Q */
            P ← Q;
            SP ← c(P);
        END
END /* of Generic_Hill_Climbing */
```

There are different ways to tackle a CSP with a hill-climbing approach

- The search space comprises the set of all possible compound labels.
- The evaluation function maps every compound label to the negation of the number of constraints being violated by it.
- Alternatively, the value of a compound label could be made the negation of the number of labels which are incompatible with some other labels in the compound label.
- Two compound labels may be considered to be adjacent to each other if they differ in exactly one label between them.
- A solution to the CSP is a set of assignments which violates zero constraints.

```
PROCEDURE Naive_CSP_Hill_Climbing(Z, D, C)
BEGIN
    /* Initialization */
    CL ← { };
    FOR each x in Z DO
        BEGIN
            v ← a random value in D_x;
            CL ← CL + {<x,v>};
        END
    /* Hill-climbing: other termination conditions may be added to pre-
        vent infinite looping */
    WHILE (the set of labels in CL violates some constraints) DO
        BEGIN
            <x,v> ← a randomly picked label from CL which is incom-
                patible with some other labels in CL;
            v' ← any value in D_x such that CL – {<x,v>} + {<x,v'>} vio-
                lates no more constraints than CL;
            CL ← CL – {<x,v>} + {<x,v'>};
        END
END /* of Naive_CSP_Hill_Climbing */
```

The problem with hill-climbing algorithms in general is that they do not guarantee successful termination. They may settle in local optima, where all adjacent points are worse than the current focal point, though the current focal point does not represent a solution. They may also loop in *plateaus*, where a number of mutually-adjacent points all have the same value.

Sometimes, additional termination conditions are added (to the WHILE loop in the Hill_Climbing algorithm above). For example, one may want to limit the number of iterations or the program's run time.

Even when hill-climbing algorithms terminate, they are not guaranteed to be efficient. But when good heuristics are available, which could be the case in some problems, hill-climbing does give us hope to solve intractable CSPs.

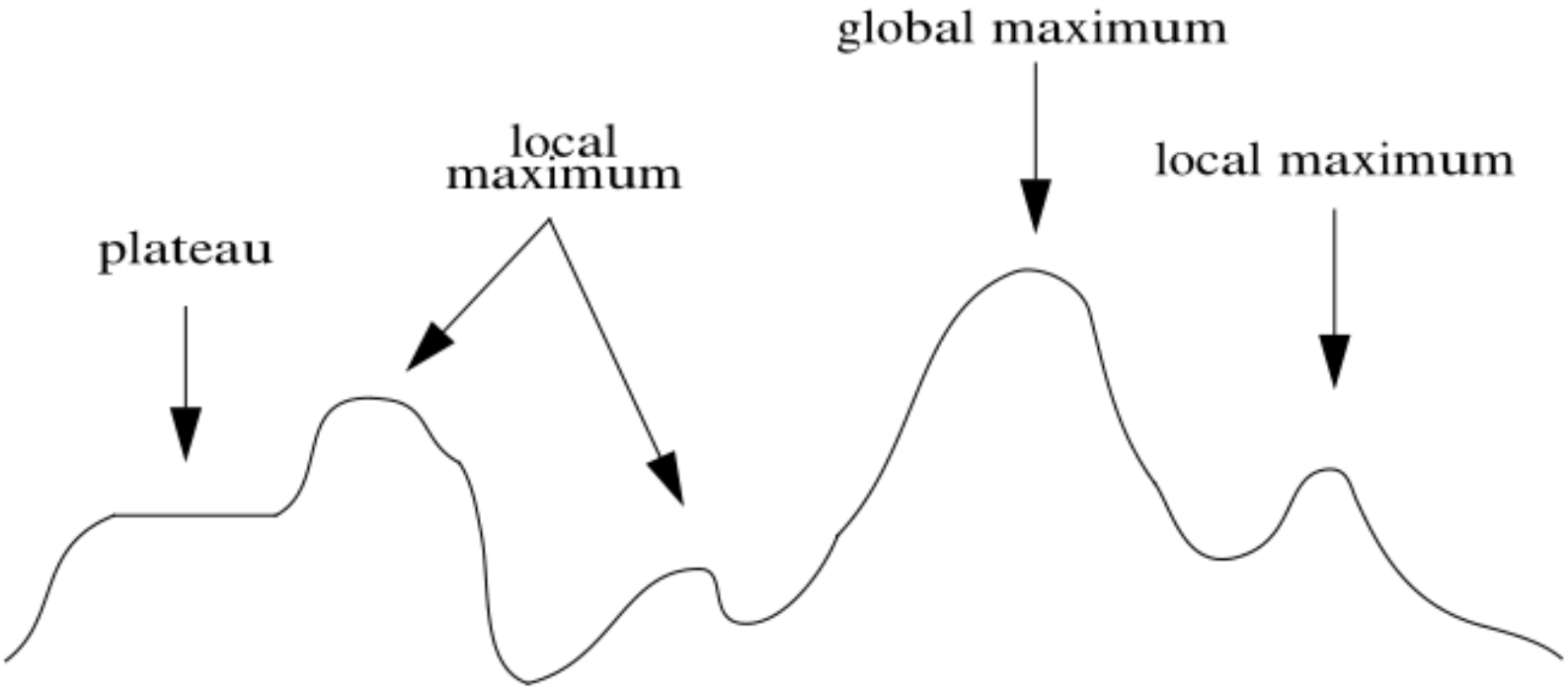


Figure 8.1 Possible problems with hill-climbing algorithms: the algorithms may stay in plateaus or local maxima

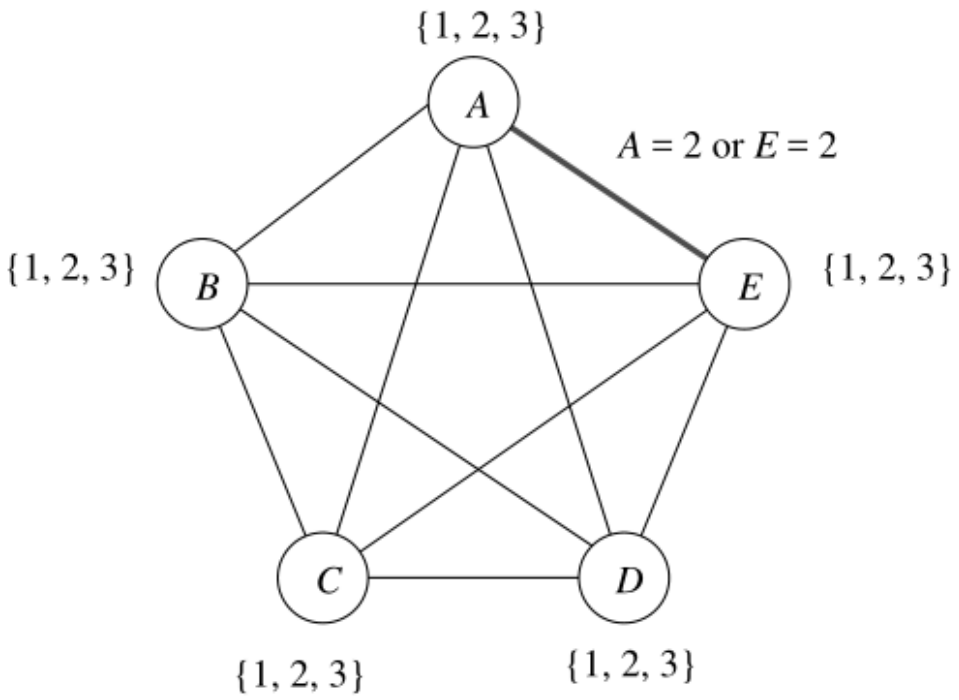
8.2.2 The heuristic repair method

The *heuristic repair method* is a hill-climbing method based on the min-conflict heuristic. It improves over the Naive_CSP_hill_climbing algorithm in the way in which it chooses the values. When a label which violates some constraints is picked for revision, the value which violates the least number of constraints is picked. Ties are resolved randomly.

```
PROCEDURE Heuristic_Repair(Z, D, C)
/* A hill-climbing algorithm which uses the Min-Conflict heuristic */
BEGIN
```

```
    /* Part 1: Initialization */
    CL ← { };
    FOR each x ∈ Z DO
        BEGIN
            V ← the set of values in D_x which violate the minimum
                number of constraints with labels in CL;
            v ← a random value in V;
            CL ← CL + {<x,v>};
        END
    /* Part 2: Hill-climbing */
    WHILE (the set of labels in CL violates some constraints) DO
    /* additional termination conditions may be added here */
        BEGIN
            <x,v> ← a randomly picked label from CL which is incom-
                patible with some other labels in CL;
            CL ← CL – {<x,v>};
            V ← the set of values in D_x which violates the minimum
                number of constraints with the other labels in CL;
            v' ← random value in V;
            CL ← CL + {<x,v'>};
        END
END /* of Heuristic_Repair */
```

The HRM has the usual problem of incompleteness in hill-climbing algorithms. Example (Figure 8.2): the HRM fails to produce a solution in most attempts. There exists only one solution, which is to have all variables labelled to 2. The HRM will normally fail to find this solution because unless three or more variables are initialized to 2, most variables will end up with values 1 or 3 and the HRM will wander around in a plateau of local minima. For example, assume that the initialized compound label is (<*A*,2><*B*,1><*C*,1><*D*,1><*E*,2>). Six constraints are violated: $C_{A,B}$, $C_{A,C}$, $C_{A,D}$, $C_{B,E}$, $C_{C,E}$ and $C_{D,E}$. The number of constraints violated can be reduced if the value of either *A* or *E* is changed to 1 or 3 (in either case, only four constraints will be violated). Even if one of *B*, *C* or *D* is picked for revision, changing its value to 2 does not reduce the number of violations, and therefore, there is a 2/3 chance that the values 1 or 3 will be picked (which does not bring the algorithm closer to the solution). If the initialized compound label has two or less 2's assigned to the variable, and *A* and *E* are not both labelled with 2, then the HRM will change between states in which the five variables take values 1 or 3, which always violate one constraint, namely $C_{A,E}$.



(All constraints C_{xy}, with the exception *of* C_{AE}, require that $x + y$ is even; C_{AE} requires that at least one of A and E takes the value 2)

Figure 8.2 Example of a CSP in which the Heuristic Repair Method would easily fail to find the only solution where all variables are assigned the value 2

8.2.3 A gradient-based conflict minimization hill-climbing heuristic

The *gradient-based conflict minimization* (*GBCM*) *heuristic* is applicable to CSPs where all variables have the same domain and the size of this domain is the same as the number of variables, and where each variable must take a unique value. It has been found to be effective in the N-queens problem, although its effectiveness in other problems is unknown.

An algorithm which uses the GBCM heuristic hill-climbs from a random compound label. If there exist two labels in the compound label which are in conflict with other labels, the values of them will be swapped when the compound label after this swap violates fewer constraints.

```
PROCEDURE QS1(n)
/* n is the number of queens in the N-queens problem */
BEGIN
    /* initialization */
    FOR i = 1 to n DO
        Q[i] ← a random column which is not yet occupied;
    /* hill-climbing */
    WHILE conflict exists DO
        BEGIN
            find any i, j, such that Q[i], Q[j] are in conflict with some
                queens;
            IF (swapping values of Q[i], Q[j] reduces the total number of
                conflicts)
            THEN swap the values of Q[i] and Q[j];
        END /* of while loop */
END /* of QS1 */
```

It is found that the initialization part of the QS1 algorithm can be improved.

```
PROCEDURE QS4(n)
    CONSTANT: c;        /* c is to be determined by the programmer */
BEGIN
    /* initialization — minimize conflicting queens */
    FOR i = 1 to n – c DO
        place a queen in a random position which does not have con-
            flict with any queen which has already been placed; if failed,
            exit reporting failure;
    FOR i = 1 to c DO
        place a queen in a random column which is not yet occupied;
    /* hill-climbing */
    WHILE (conflict exists) DO
        BEGIN
            find any i, j, such that Q[i], Q[j] are in conflict with some
                queens;
            IF (swapping values of Q[i], Q[j] reduces the total number of
                conflicts)
            THEN swap the values of Q[i] and Q[j];
        END /* of while loop */
END /* of QS4 */
```

8.3 Connectionist Approach

8.3.1 Overview of problem solving using connectionist approaches

The *min-conflict heuristic* is derived from a connectionist approach. A connectionist approach uses networks where the nodes are very simple processors and the arcs are physical connections, each of which is associated with a numerical value, called a *weight*. At any time, each node is in a state which is normally limited to either *positive* (*on*) or *negative* (*off*). The state of a node is determined locally by some simple operations, which take into account the states of this node's directly connected nodes and the weights of those connecting arcs. The *network state* is the collection of the states of all the individual nodes. In applying a connectionist approach to problem solving, the problem is represented by a connectionist network. The task is to find a network state which represents a solution.

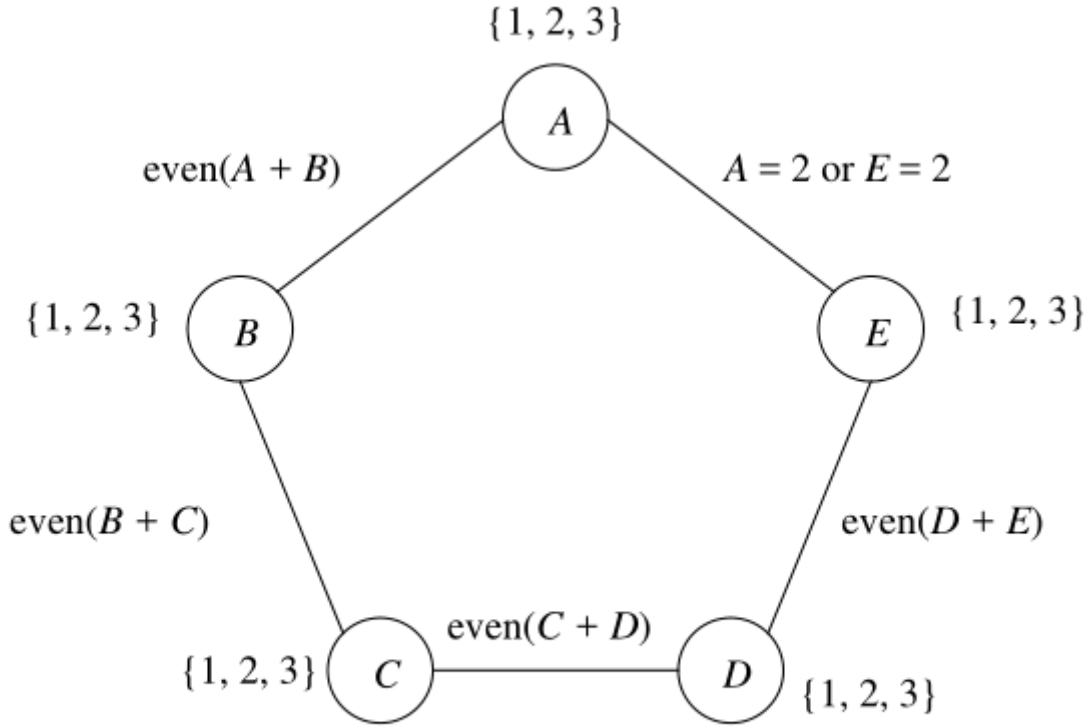
8.3.2 GENET, a connectionist approach to the CSP

The network is initialized by assigning -1 to all the weights. One arbitrary node per cluster is switched on, then the network is allowed to converge under the following rule. The input to each node is computed by the rule:

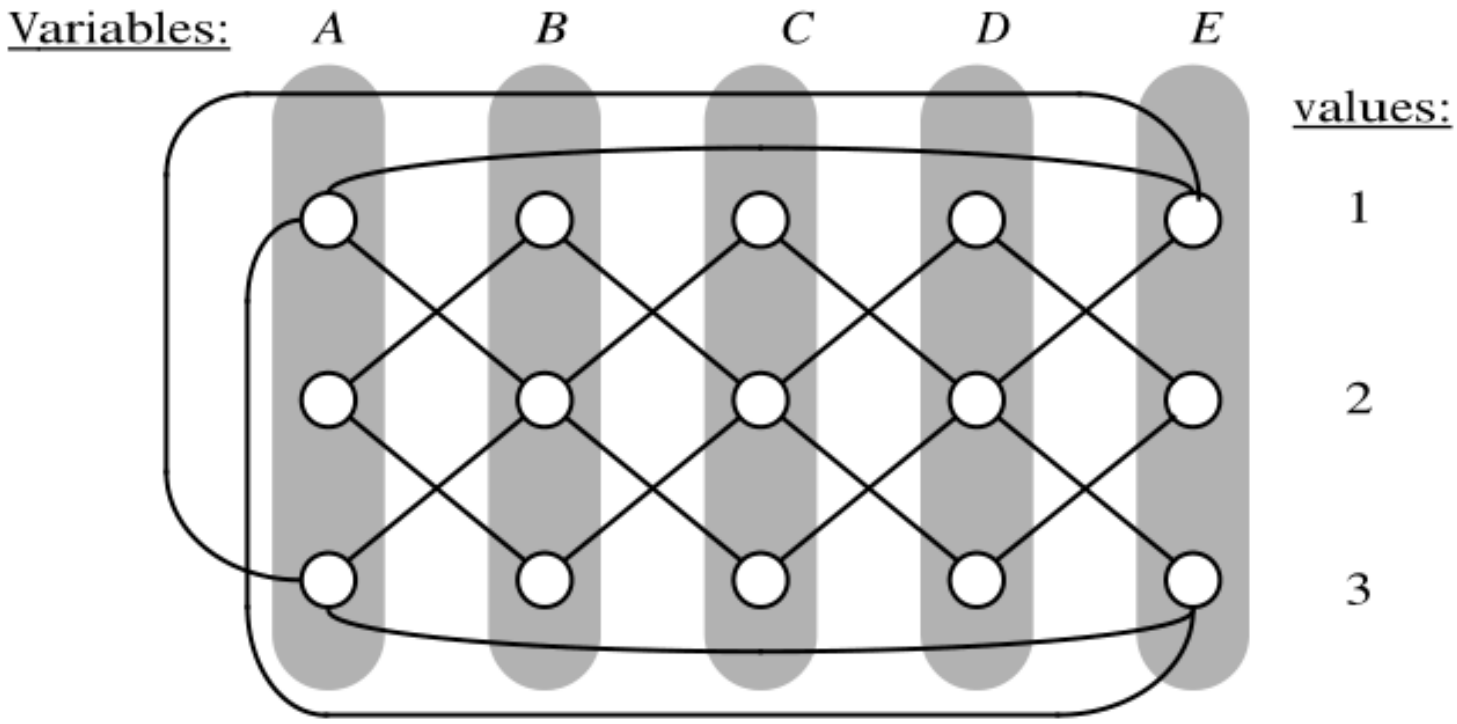
$$\text{input of } x = \sum_{y \leftarrow adjacent\,(x,\,y)} w_{x,\,y} \times s_y$$

where $w_{x,y}$ is the weight of the connection between nodes x and y, and s_y is the state of y, which is 1 if y is on and 0 if y is off. So the input to a node x is the sum of the weights on the connections which connect x to nodes that are *on* at the point of calculation. The nodes in each cluster continuously compete to be turned on. In every cluster, the node with maximum input will be turned on, and the rest will be turned off. Since there are only connections with negative weights, the winner in each cluster is a label violating the fewest constraints for the subject variable. In tie situations, if one of the nodes in the tie was already on, will stay on, otherwise a random choice is made to break the tie.

Figure 8.3 Example of a binary CSP and its representation in GENET



(a) Example of a binary CSP (variables: A, B, C, D and E)



(b) Representation of the CSP in (a) in GENET all connections have their weights initialized to −1

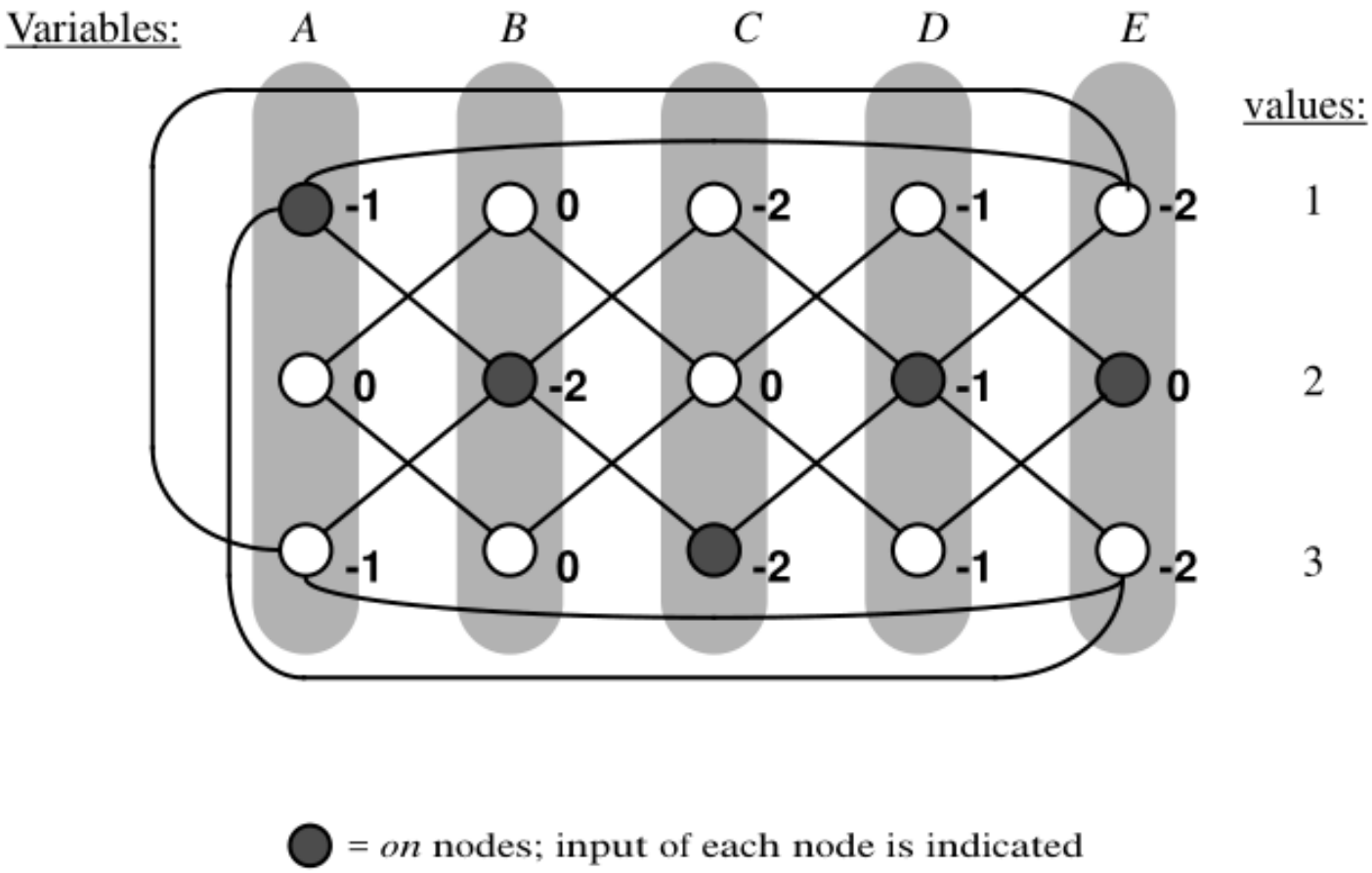


Figure 8.4 Example of a network state in the GENET network shown in Figure 8.3(b) (all connections have weights equal to −1)

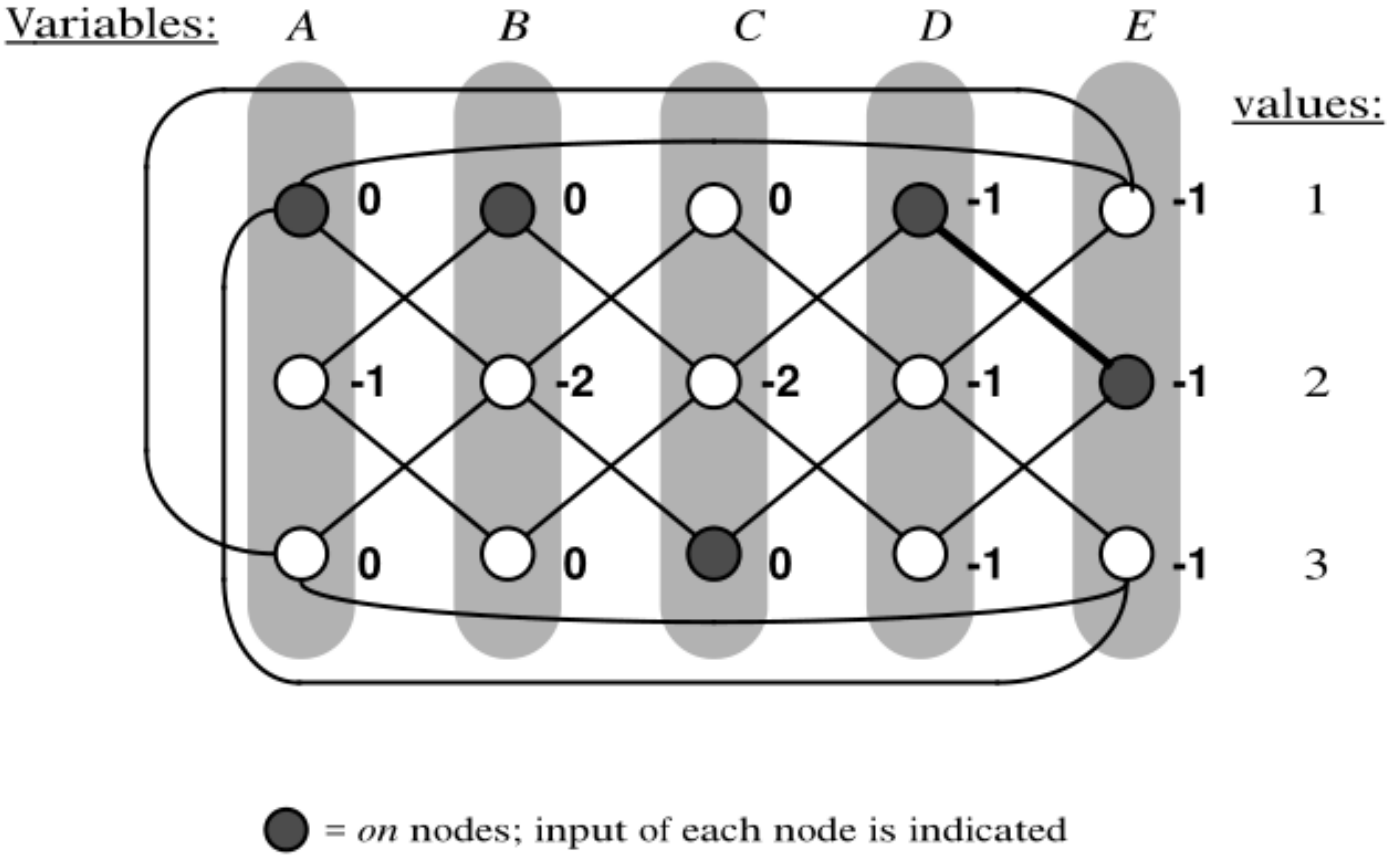


Figure 8.5 Example of a converged state in the GENET network shown in Figure 8.3(b) (all connections have weights equal to −1)

Figure 8.4 shows a state of the network shown in Figure 8.3. The cluster of nodes which represent the labels for variable *A* is unstable because the node which represents <*A*,2> has the highest input (0) but is not switched on. Similarly, the clusters for *B* and *C* are unstable because the *on* nodes in them do not have the highest input. Cluster *D* is stable because the node which represents <*D*,2> has an input of −1, which is a tie with the other two nodes in cluster *D*. According to the rules described above, the node representing <*D*,2> will remain *on*. There is no rule governing which of the clusters *A*, *B* or *C* should change its state next, and this choice is non-deterministic. The change of state in one cluster would change the input to the nodes in other clusters. For example, if the node for <*A*,1> is switched *off*, and the node for <*A*,2> is switched *on*, the input of all the three nodes in cluster *B* would be −1, which means cluster *B* would become stable. When the network settles in a stable state (called a *converged* state), GENET will check to see if that state represents a solution. In a converged state, none of the *on* nodes have lower input than any other nodes in the same cluster. A state in which all the *on* nodes have zero input represents a solution.

Otherwise, the network state represents a local minimum. The converged state in Figure 8.5 represents a local minimum because the inputs to the nodes which represent <*D*,1> and <*E*,2> are both −1. When the network settles in a local minimum, the state updating rule has failed to use local information to change the state. When this happens, the following heuristic rule is applied to remove local minima:

$$\text{New } w_{ij} = \text{Old } w_{ij} - s_i \times s_j$$

The local minima is removed by decreasing the weights of violated connections (constraints). This simple "learning" rule effectively does two things: it reduces (continuously if necessary) the value of the current state until it ceases to be a local minima. Besides, it reduces the possibility of any violated constraint being violated again. The hope is that after sufficient "learning" cycles, the connection weights in the network will lead the network states to a solution.

In Figure 8.5, the weight on the connection between the nodes <*D*,1> and <*E*,2> will be decreased by 1 to become −2. This will make the input to both nodes for <*D*,1> and <*E*,2> −2. So, the state of either cluster *D* or *E* will be changed.

```
PROCEDURE GENET
BEGIN
    One arbitrary node per cluster is switched ON;
    REPEAT
        /* network convergence: */
        REPEAT
            Modified ← False;
            FOR each cluster C DO IN PARALLEL
                BEGIN
                    On_node ← node in C which is at present ON;
```

```
                Label_Set ← the set of nodes within C which input
                    are maximum;
                IF NOT (On_node in Label_Set) THEN
                    BEGIN
                        On_node ← OFF;
                        Modified ← True;
                        Switch an arbitrary node in Label_Set to ON;
                    END
            END
    UNTIL (NOT Modified);       /* the network has converged */

    /* learn if necessary: */
    IF (sum of input to all ON nodes < 0)
            /* network settled in local maximum */
    THEN FOR each connection c connecting nodes x & y DO IN
        PARALLEL
                IF (both x and y are ON)
                THEN decrease the weight of c by 1;
  UNTIL (input to all ON nodes are 0) OR (any resource exhausted)
END /* of GENET */
```

8.3.3 Completeness of GENET

There is no guarantee of completeness in GENET, as can be illustrated by the simple example in Figure 8.6.

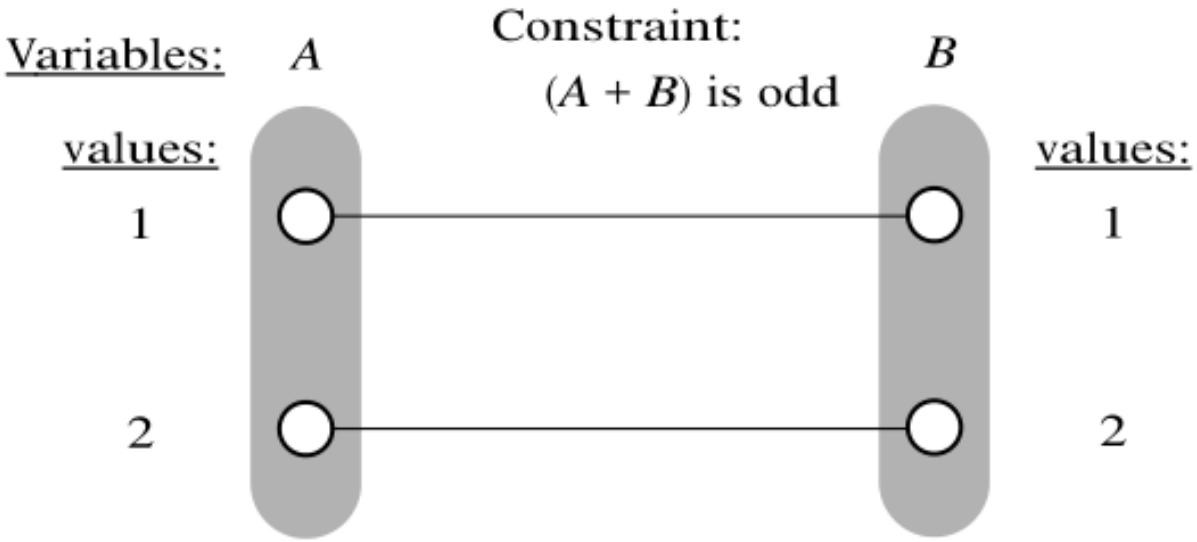


Figure 8.6 Example of a network in GENET which may not converge (the network may oscillate between (<*A*,1> <*B*,1>) and (<*A*,2> <*B*,2>))